

Travelogix

Making Travels Easy

Github Link : [GitHub Repository](#)

Team Member

Ramisa Anan Rahman ID: 220042105

Ridika Naznin ID: 220042115

Afrin Jahan Era ID: 220042132

Supervisor

Farzana Tabassum

Lecturer, Department of CSE

Overview

Travelogix is a console-based travel management application developed in Java which is designed to simplify the whole travel planning process. It is mainly built to make a hectic travel to a simple and fun one .

The application facilitates trip organization through features like destination selection, budget tracking, transportation management and itinerary planning confirming strong authentication and privacy. Instead of relying on multiple applications for managing different activities tracking for travelling , users can manage their entire journey from a single platform.

The system serves three primary user roles with distinct functionalities:

- **Travelers:** By Using this application Travelers can have a personalized experience for managing their destinations, schedules, budgets, notes and memories of travels.
- **Travel Agencies:** The main responsibilities of Travel Agencies are to assist users by offering transport details and seat assignments. Here they only do work to keep the journey of a user very easy by booking their seats according to choices .
- **Administrators:** Admins maintain the integrity of the system by monitoring the whole system and handling agency requests.

By integrating core travel management features into a single, structured interface, Travelogix provides an efficient alternative to fragmented planning tools.

Motivation

The development of Travelogix was driven by clear gaps which are observed in current travel planning methodologies. Our research identified three fundamental challenges in existing solutions:

1. Confusing or broken travel planning process

As we can see, the existing travel management requires users to navigate multiple disconnected platforms for itinerary creation, transportation booking, and expense tracking. This fragmentation leads to:

- Travelers have to think more
- More chances to miss important details
- Poor use of time when planning trips

2. Money tracking is scattered and messy.

Most travelers currently maintain separate systems for:

- Transportation costs
- Accommodation expenses
- Daily activity budgets

This makes it hard to manage money and leads to mistakes.

3. Poor Traveler-Agency Coordination

The absence of a centralized communication platform between travelers and service providers results in:

- Delayed information updates
- Booking inconsistencies
- Suboptimal resource utilization



Travelogix solves these challenges by offering:

- **Unified all system needed for travelling**

It offers a centralized platform integrating all critical travel management functions—transports, itineraries, and budgets—into a single, intuitive interface.

- **Real-Time Synchronization**

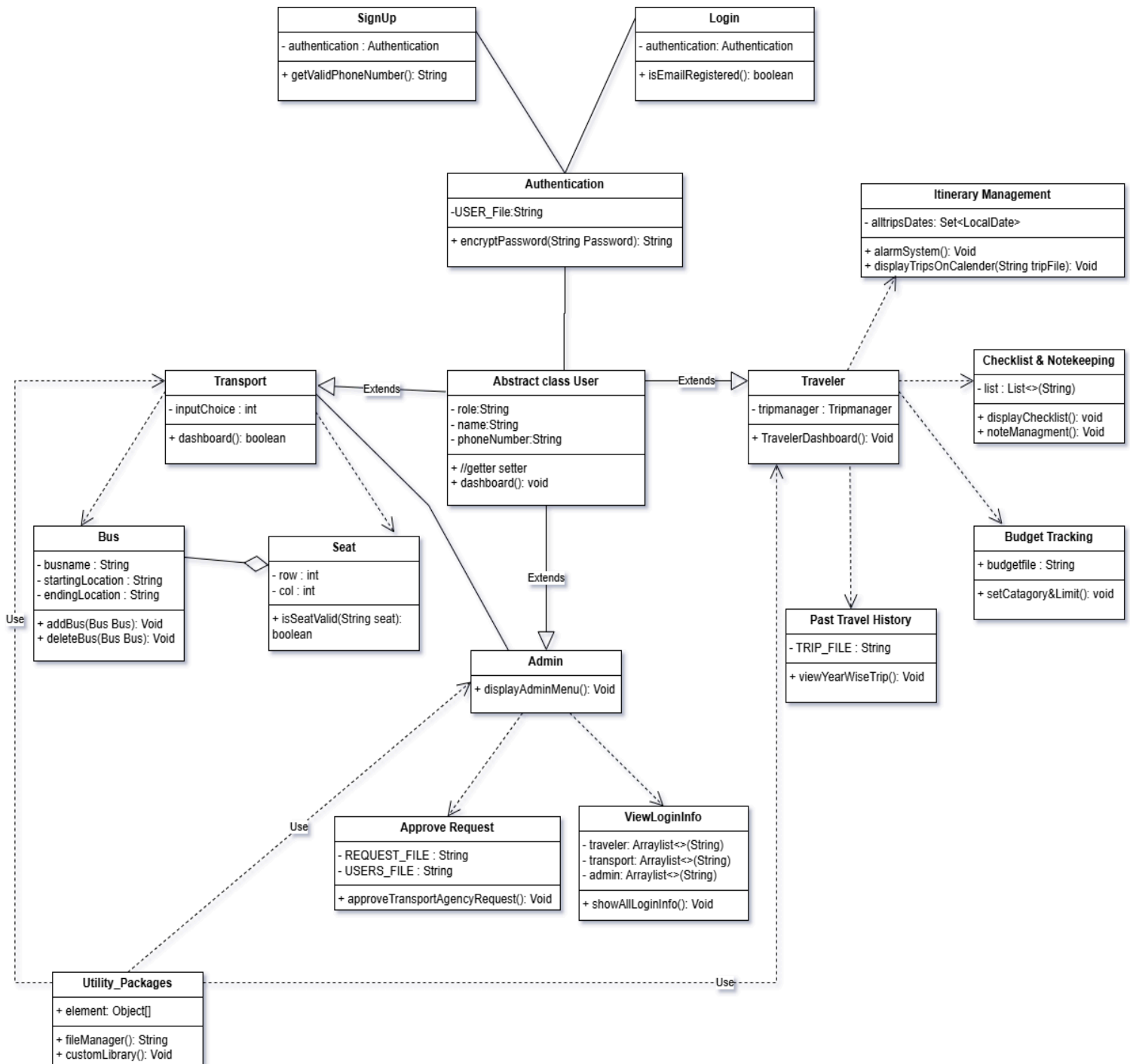
Travelogix also offers a real time reminder system to keep an update of the traveling date to travelers.

- **Customized Role-Based Workflows**

Purpose-built interfaces for travelers (simplified planning), agencies (for managing seats and routes for travelers), and administrators (control these systems like transport agencies delete account requests).

So it was the main motivation for us to build this travel application to turn a messy travel into a simple one.

Class Diagram





OOP Relationships in the UML Diagram

1. Inheritance (Generalization)

- Traveler → User (Child class inherits from User)
- Admin → User (Admin is a specialized User)

2. Association (Uses Relationship)

- SignUp & Login → Authentication (Both use Authentication for validation)
- Traveler → TripManager (Implied dependency for trip management)

3. Aggregation (Weak "Has-A")

- Bus → Seat (Bus contains seats, but seats can exist independently)

4. Dependency (Temporary Use)

- Other classes depend on Utility_Packages for file operations and custom libraries.

5. Abstraction Relationships

- abstract classes shown (e.g., User is abstract class)

Key Features (Use-Case Descriptions)

1. Location Selection

Travelers can browse and select their desired destinations, initiating the trip planning process. Here travelers can select their location with starting and ending location and also can browse location according to the desired location.

2. Transport Booking & Seat Assignment

Transport agencies add routes and manage seats of their transports . Here travelers will contact the transport agencies and will tell them about booking seats and transport agencies will manage seats and book for them according to travelers requirements .

Reserve Seat

Enter the number plate of the bus to book seats: gaz-00

Seat matrix before booking:

SEATS

Seat matrix:

[X] [X] [1C] [1D]

[2A] [2B] [2C] [2D]

[3A] [3B] [3C] [3D]

[4A] [4B] [4C] [4D]

[5A] [5B] [5C] [5D]

[6A] [6B] [6C] [6D]

How many seats would you like to book? 2

Enter seat (e.g., 1A, 2B): 1A

Seat already booked. Please select a different seat.

Enter seat (e.g., 1A, 2B): 1c

Seat 1C booked!

3. Checklist & Note Keeping

To assist travelers in organizing their trips, the system includes a dynamic packing checklist. Users can add, modify, or remove items as needed. Additionally, they can write or record notes related to their journey along with voice notes which ensure important details are easily accessible.

```
--- Checklist ---  
1. (.) blanket  
2. (X) t-shirt  
3. (.) notebook  
Total items: 2
```

4. Itinerary Management

Users can set travel dates and set alarms to stay on schedule. Also, the system displays trips in a calendar view, with trip dates highlighted in red and overlapping dates marked with a blue 'XX' sign. Notifications with alarms ensure travelers never miss critical events.

Year: 2025

JANUARY 2025							MARCH 2025							SEPTEMBER 2025						
Sun	Mon	Tue	Wed	Thu	Fri	Sat	Sun	Mon	Tue	Wed	Thu	Fri	Sat	Sun	Mon	Tue	Wed	Thu	Fri	Sat
			1	2	3	4							1		1	2	3	4	5	6
5	6	7	8	9	10	11	2	3	4	5	6	7	8	7	8	XX	XX	XX	XX	XX
12	13	14	15	16	17	18	9	10	11	12	13	14	15	XX	XX	XX	XX	XX	19	20
19	20	21	22	23	24	25	16	17	18	19	20	21	22	21	22	23	24	25	26	27
26	27	28	29	30	31		23	24	25	26	27	28	29	28	29	30				
							30	31												

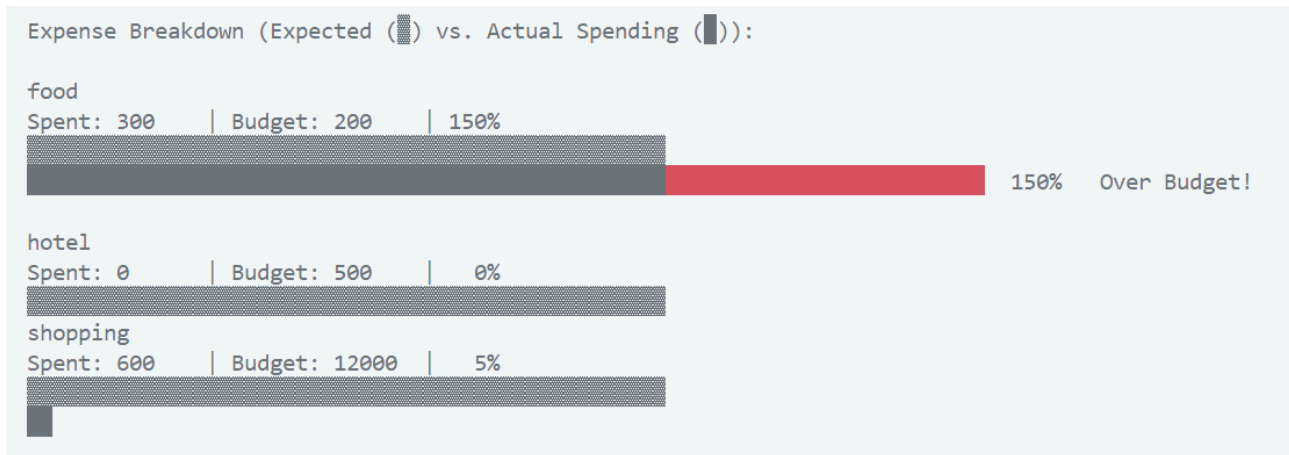
OCTOBER 2025						
Sun	Mon	Tue	Wed	Thu	Fri	Sat
			1	2	3	4
5	6	7	8	9	10	11
12	13	14	15	16	17	18
19	20	21	22	23	24	25
26	27	28	29	30	31	

5. Past Travel History

The system generates summaries of past trips, including visited destinations, budget utilization etc. These insights enable travelers to analyze their travel patterns and make better decisions for future trips.

6. Budget Tracker

Travelers can input expenses and compare them against predefined budgets. Expenses are categorized and visualized using ASCII-based graphs, where exceeded amounts are highlighted in red for quick identification. This feature helps users maintain financial control during their trips.



7. Admin Panel

Administrators oversee system operations by managing travel agency requests, maintaining system integrity. They have the authority to approve or reject account deletion requests from transport agencies, ensuring only verified information is displayed. It is important because transport agencies are under a job, so their contact will be given to travelers for transport related information, so the transport agencies need permission to delete their account from admin. So admin confirms that travelers will only get the contact info of the present transport agency's person.

Common Functionalities

Authentication

Before accessing Travelogix, users must authenticate their accounts. Whether they are travelers, transport agencies, or admins, each user must log in using credentials such as a username, email, and password. The system employs password masking and encryption to protect sensitive data.

```
===== LOG IN =====  
Enter your email      : admin@gmail.com  
Enter your password  :*****
```

Account Management

All users—travelers, transport agencies, and admins—can manage their accounts within the system. They can:

- View personal details (name, contact information, etc.).
- Delete accounts if they no longer wish to use the service.

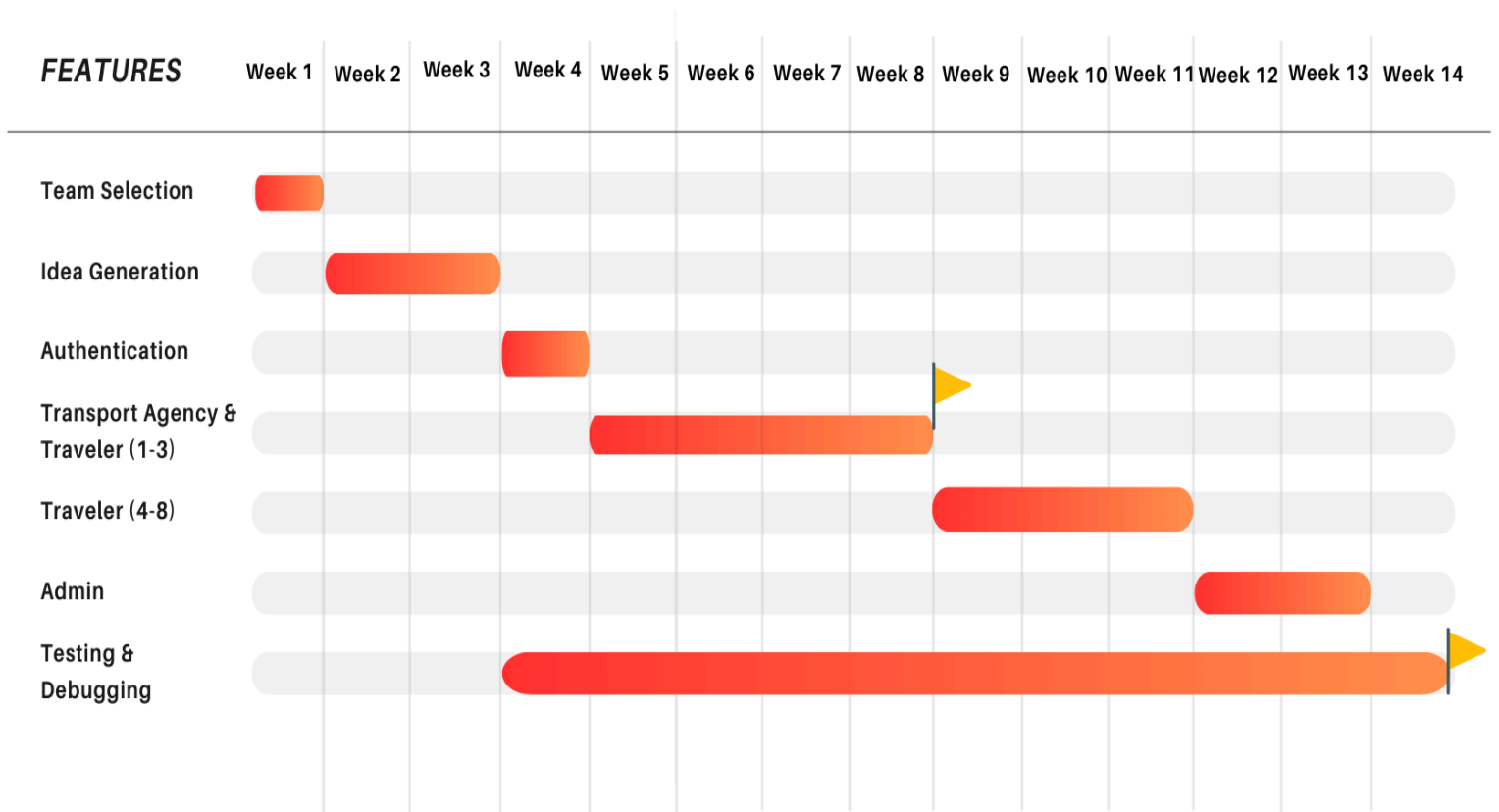
Tools and Technologies Used

Tool	Purpose of Using
Java	Java served as the core programming language for backend, offering strong object-oriented principles and scalability to support the application's layered architecture.
IntelliJ IDEA / VS Code	Used as primary IDEs, enhancing productivity with intelligent coding assistance, debugging, and seamless integration with development tools.
Git & GitHub	Git & GitHub enabled efficient version control and collaborative development, ensuring smooth teamwork and code management throughout the project lifecycle.
Draw.io	It helped to visualize class diagrams and workflows, streamlining design discussions and maintaining clear documentation.
TXT File Storage	TXT File Storage provided a lightweight, straightforward solution for data persistence, aligning with the project's simplicity and ease of implementation.

Alternatives Considered

- GUI with JavaFX instead of console
- MySQL instead of TXT files
- For OOP based language with .net framework C#

PROJECT TIMELINE



Detailed Contribution of Each Member

Member	Contribution
Ramisa Anan Rahman ID: 220042105	Implemented authentication, itinerary features and modified budget tracking feature; assisted in combining traveler's features.
Ridika Naznin ID: 220042115	Developed budget tracking logic and entire transport agency system including managing and updating all bus details.
Afrin Jahan Era ID: 220042132	Did work on Admin part , Checklist & Note Keeping , Past Travel History and integrated the whole coding part done by three team members together .

External Library/Package

Library/Package	Purpose
javax.sound.sampled	Audio playback/recording (WAV, AIFF, etc.).
java.security.MessageDigest	Hash passwords (SHA-256, MD5).
java.math.BigInteger	Convert binary hash data to hexadecimal strings.
org.junit.jupiter (JUnit 5)	Write test cases (@Test, assertions).
java.time.LocalDate	Modern date/time API (LocalDateTime, ZonedDateTime, formatting).

Custom Library Function

1. Custom ArrayList
2. String Operations - trim() , equals() , length()
3. customHash()

Challenges Faced

- **Data Persistence:** As we used TXT files ,so without corrupting user data or introducing concurrency bugs required careful design and rigorous testing. Testing became a little difficult because of dependencies .
- **Role-Based Logic:** Designing and maintaining a clean role-based system that gave each user (Traveler, Agency, Admin) access to only their relevant functionalities required thoughtful separation of concerns and interface usage.
- **Console Interface Limitation:** Without a graphical UI, it was difficult to present a user-friendly interface. We had to rely on formatting tricks, menus, and color coding to simulate clarity and interaction.
- **Password Masking in Console:** Implementing secure password masking (displaying asterisks instead of plain text while typing passwords) in Java console applications was a major challenge. The standard Scanner and BufferedReader classes do not support hiding input. After multiple failed attempts, we implemented a custom solution using console manipulation via the EraserThread class. This required synchronization to prevent flickering and ensured input was captured securely without echoing characters.
- **Colored Calendar Misalignment:** While highlighting trip dates in red on the calendar improved clarity, the use of ANSI escape codes for color caused line misalignment due to invisible character lengths. Managing consistent spacing and alignment across days of the week became tricky, especially in months with multiple marked dates. We resolved this by fine-tuning spacing logic and compensating for the length of color codes.
- **Merge Conflicts & Integration:** As multiple features were developed simultaneously by different team members thats why integrating modules and managing merge conflicts—especially for shared files like User and Trip—required regular coordination and version control discipline.

Key Learnings from the Project

1. **OOP Principles in Practice** – Effectively applied inheritance and abstraction to streamline code structure. It gives us a more clear understanding of oop .
2. **Modular Design & Teamwork** – Breaking tasks into smaller modules improved efficiency and allowed seamless collaboration across the team.
3. **Version Control with Git** – Gained hands-on experience in real-world Git workflows, ensuring smooth code integration and conflict resolution.
4. **Early Testing & Debugging** – We have done so many unit testing and debugging which makes a clear understanding for us about TDD .
5. **Security Implementation** – Incorporated password masking in the console to enhance user privacy and security.
6. **SOLID** – Although we were unable to implement all the 5 principles of SOLID as it was our first learning about SOLID, we have applied OCP, SRP etc. While implementing features it became more clear how SOLID works and will help us to do code following the gaps in our future project .

While working on the project , we also have to realize where our code is messy which gives us understanding of clean code .

Individual Reflections

Ramisa Anan Rahman - 220042105

In this project, I worked on the role-based authentication system by validating email and phone number formats, implementing password masking and encryption for secure login, and ensuring smooth user management. I also developed the Itinerary Management module, which included a color-coded calendar view for trips and alarm/notification reminders for important events.

Additionally, although the initial Budget Tracker was designed by Ridika, I modified and enhanced it. I added features such as displaying red bars when the budget is exceeded, showing percentages next to each category, triggering alarms and notifications upon overspending, and highlighting exceeded amounts in red within the budget table.

Finally, I created the Traveler Dashboard to integrate all travel features into a user-friendly console.

What worked well was our clear division of tasks and regular syncing through GitHub. However, merging modules sometimes caused integration issues. We overcame these by setting weekly meetings and conducting frequent code reviews.

Through this project, I learned the importance of detailed communication, early planning, and adaptability within a team. Regular collaboration, version control, and proactive testing. Going forward, I will focus even more on early integration testing and clearer documentation to improve teamwork efficiency.



Ridika Naznin - 220042115

I was responsible for developing the entire Transport Management module, including adding/deleting buses, updating bus details, searching buses by number plate, showing seat matrices, and implementing the seat booking system. Additionally, I designed the initial Budget Tracker logic, which involved setting categories and spending limits, and dynamically adding expense rows as users input data including the remarks.

This work required extensive file handling operations in Java using CSV format, which I learned through this project. I collaborated closely with my teammates, Ramisa and Era, which taught me the importance of regular communication and modular code development.

Another thing I learnt through this project is maintaining version control with github, which is one of the most important things in project collaborations. I learnt a lot from my team members through this process.

One of the major challenges was ensuring file consistency while updating transport and budget records. Through this project, I enhanced my understanding of backend system design, unit testing, file management, and team collaboration.

Moving forward, I plan to focus even more on early code integration and error handling for better project efficiency.

Afrin Jahan Era - 220042132

- I worked on 3 parts of the features which are - Admin part , Checklist & Notekeeping , Past Travel History .

For Admin Part I did work on the request approval from transport agencies along with account deletion of transport agencies profile . Then I worked on Checklist and Notes . For the checklist I have done adding the checklist using (.) and (X) . For the voice part I have integrated a sampled library to add the feature of voice notes with dynamically stopped timer of voice notes and written notes with multiline notes finishing a word with DONE . then for past travel history here i showed past travel visited cities and areas with date and time along with their spending budgets which is integrating with budget tracker.

Then last of all I connected all the code together done by my team mates and organized the code .

- Working on the checklist and notes feature was enjoyable, especially since it was my first time implementing voice notes in a console-based application. However, connecting all the code was initially messy—I encountered many exceptions due to dependencies. I managed to resolve them, but we later tried applying clean code principles to make integration and feature additions easier. Unfortunately, due to time constraints, we couldn't fully implement this, so the organization didn't go as smoothly as planned.
- I was responsible for merging branches with the main branch. Since it was our first time working as a team and everyone had different working hours, I initially faced merge conflicts. Eventually, we sat together and figured out a system—pushing changes one by one—to make merging easier. My teammates' coordination helped me keep up with the teamwork.
- Coordination is what made us successfully complete the project with good feedback . So in future we will apply it from the very beginning . First we will discuss what to do and who will do that one by one .

So these are my personal reflections on the project while working on it .